KB

SECURITY ASSESSMENT REPORT

API Security Mini Review

OWASP crAPI — Local Training Instance

PREPARED BY	Kyunghwan Byun
SPECIALTY	Web & API Security Researcher
DATE	June 2026
VERSION	1.0
REFERENCE	DEMO-API-2026-01

Web & API Security Researcher · YesWeHack Hunter

Document Control

Version	Date	Author	Status	Description
0.1	2026-06-08	Kyunghwan Byun	Draft	Initial draft from verified lab notes
1.0	2026-06-09	Kyunghwan Byun	Released	Portfolio release

About This Document

This document is a sample security assessment report prepared for portfolio and demonstration purposes only. The assessment was performed against a local **OWASP crAPI** training instance — an intentionally vulnerable application — and not against any real client, customer, or production system. It contains no confidential third-party information and may be shared freely.

AI-assisted analysis was used for endpoint classification, parameter mapping, and vulnerability-candidate triage. That output was treated as a starting point; every reported finding was manually reproduced and verified before inclusion.

Contents

Executive Conclusion

Testing against the local crAPI lab confirmed one access-control issue in the vehicle location API. With a normal authenticated account, a tester could request a different user's vehicle location by reusing that user's vehicle UUID in `GET /identity/api/v2/vehicle/{vehicleId}/location`.

The recommended fix is to enforce server-side vehicle ownership checks before returning location data and to add regression tests for owner, non-owner, unauthenticated, and nonexistent vehicle IDs.

Report attribute	Value
Target	OWASP crAPI local training instance
Assessment type	Small-scope Web/API security mini review
Environment	Local, authorized, non-production training lab
Primary focus	Authentication, authorization, access control, BOLA/IDOR, sensitive data exposure
AI usage	Endpoint classification, parameter mapping, and candidate triage only
Confirmation standard	Manual reproduction required before inclusion as a finding
Redaction status	Tokens, cookies, vehicle UUIDs, and account emails redacted from report evidence
Confirmed findings	1
CWE mapping	CWE-639: Authorization Bypass Through User-Controlled Key
CVSS v3.1 estimate	`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:N/A:N` (Base 4.3, Medium)

1. Executive Summary

This sample report documents the structure and communication style for a small-scope API security mini review of a local OWASP crAPI training instance. This report is a portfolio artifact and does not represent testing of a real company, production environment, customer system, or third-party service.

The review focused on authentication, authorization, API access control, object-level authorization, sensitive data exposure, and practical hardening recommendations. These areas map directly to common API risk categories and fit a developer-facing security review.

Manual testing reproduced one issue: a broken object-level authorization weakness in the vehicle location API. An authenticated non-owner user could request another user's vehicle location object by changing the vehicle identifier in the request path.

Summary item	Result
Authentication baseline	Protected endpoint rejected a request without an authorization header
Confirmed authorization issue	Non-owner authenticated users could retrieve another user's vehicle location
Primary impact	Cross-account exposure of location coordinates and account email field
Severity	Medium
Recommended priority	For equivalent production code, fix before release; add regression tests for object ownership checks
Honest severity expectation	Medium, because exploitation requires authentication and a valid vehicle identifier, but the exposed object contains sensitive location data

2. Scope

The scope covered API behavior observable in a local OWASP crAPI instance. The purpose was to demonstrate testing methodology and report quality, not to claim complete security coverage.

In scope:

Area	Included review activity
Authentication	Session handling and bearer-token enforcement for protected API access
Authorization	Cross-account access checks using controlled local test accounts
Access control	Ownership boundaries across user-accessible API objects
BOLA / IDOR	Object identifier substitution between controlled local test users
Sensitive data exposure	Review of excessive response fields in verified API responses
Security hardening	Practical defensive recommendations based on observed API behavior

Out of scope:

Activity	Reason
Denial-of-service testing	Destructive and unnecessary for this portfolio sample
Brute force or credential stuffing	Not needed to demonstrate API review quality
Phishing, social engineering, malware, or persistence	Outside the local API security scope
Production or third-party systems	The target was local OWASP crAPI only
Source code audit	This sample used black-box API behavior and cleaned evidence
Submission of real customer data or secrets to external AI tools	Sensitive data must remain local and redacted

3. Methodology

This review examined a local OWASP crAPI instance, focusing on authentication, authorization, API access control, object-level authorization, and sensitive data exposure.

Internal endpoint analysis tooling supported endpoint classification, parameter mapping, and vulnerability-candidate triage. That output was treated as a starting point, not as evidence of a confirmed vulnerability.

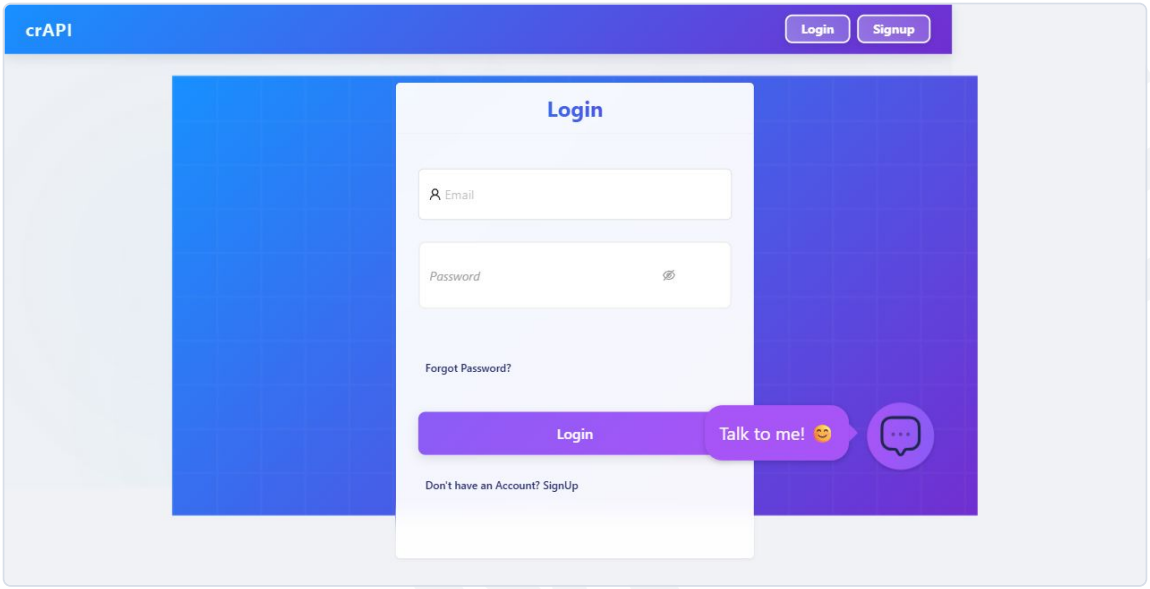
All findings included in this report were manually reproduced and verified. This review is not a full penetration test; it represents a small-scope API security review designed to demonstrate reporting style, testing methodology, and practical remediation guidance.

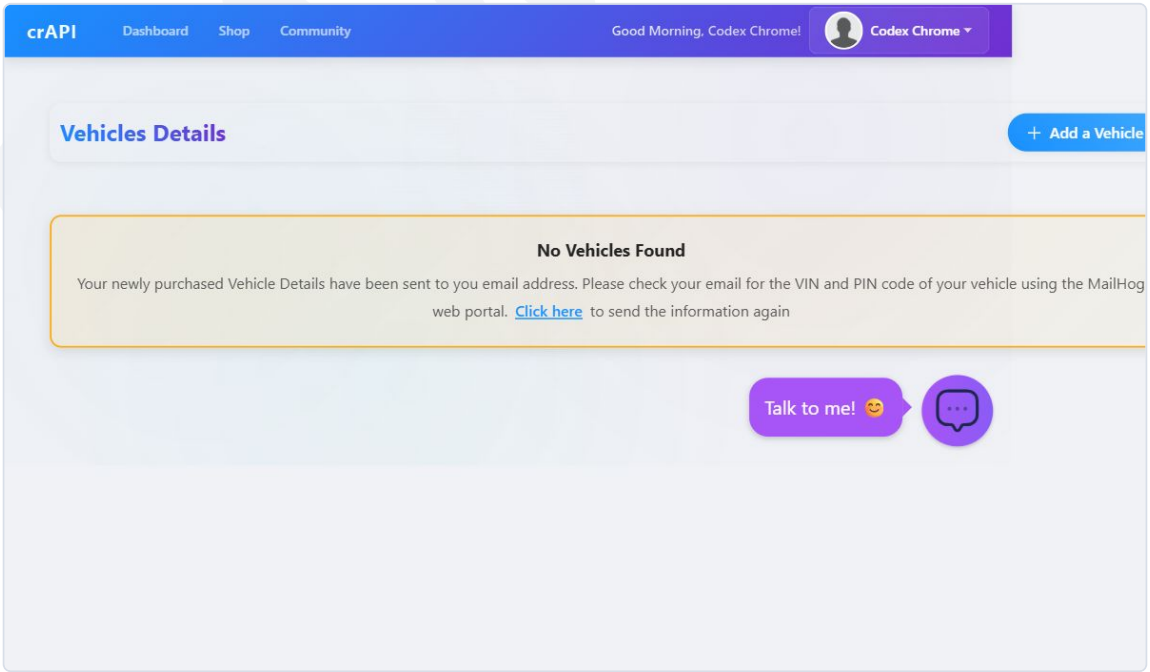
For each issue candidate, the verification workflow was:

1. Capture the relevant request and response from the local crAPI instance.

2. Identify the account, role, object identifier, and expected access boundary.
3. Reproduce the behavior manually with controlled local test accounts.
4. Compare the vulnerable response against the expected secure response using the same endpoint and object ID.
5. Save cleaned evidence: request summary, response status, response excerpt, and screenshots where useful.
6. Redact tokens, cookies, session identifiers, account emails, and object identifiers.
7. Include the issue as a finding only when the observed behavior clearly violates the expected access-control boundary.

The screenshots below show the local training application context used for the sample review. They are included only to document that the evidence came from a local crAPI lab workflow, not a production target.





4. Tested Areas

The table below separates verified coverage from areas that remained outside the small time-box.

Tested area	Review objective	Evidence status
Authentication	Determine whether protected API access requires authorization	Verified: request without authorization returned `401`
Authorization	Confirm that users cannot access resources outside intended permissions	Issue confirmed in vehicle location access
Object-level authorization	Test whether changing object IDs allows access to another local user's object	Confirmed for vehicle location UUID
Object property authorization	Check whether responses expose fields that the caller should not receive	Not tested independently; excess fields (location, email) were observed within the BOLA response
Function-level authorization	Check whether privileged API functions reject non-privileged users	Not covered in this mini review
Sensitive data exposure	Identify unnecessary user, vehicle, order, coupon, or internal metadata in responses	Observed within the single BOLA finding (coordinates + email); not tested as an independent issue
Security misconfiguration	Review headers, error handling, debug exposure, and overly verbose responses	Not covered in this mini review

5. Findings Summary

Finding ID	Title	OWASP API Mapping	Severity	Status
CRAPI-API-001	Broken object-level authorization exposes another user's vehicle location	API1:2023 Broken Object Level Authorization	Medium	Confirmed

The findings summary lists only manually reproduced issues. Candidate issues without complete evidence should remain outside the confirmed findings section until the endpoint, role boundary, expected behavior, observed behavior, impact, and cleaned evidence are all available.

6. Detailed Findings

Finding ID: CRAPI-API-001

Field	Value
Title	Broken object-level authorization exposes another user's vehicle location
Severity	Medium
Status	Confirmed
OWASP Mapping	API1:2023 Broken Object Level Authorization
CWE Mapping	CWE-639: Authorization Bypass Through User-Controlled Key
CVSS v3.1 Estimate	`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:N/A:N` (Base 4.3, Medium)
Affected Endpoint	`GET /identity/api/v2/vehicle/{vehicleId}/location`
Affected Roles / Accounts	Authenticated local crAPI user accounts

DESCRIPTION

The vehicle location endpoint accepts a vehicle UUID in the request path and returns location data for that vehicle. During manual verification, the endpoint correctly rejected a request without an authorization header. However, after authenticating as non-owner user accounts, the same endpoint returned another user's vehicle location when the request path supplied that vehicle UUID.

This indicates that the endpoint checks authentication, but does not enforce an object ownership check between the authenticated user and the requested vehicle object.

IMPACT

An authenticated user can retrieve another user's vehicle location data by supplying that user's vehicle identifier. In the verified response, the API returned vehicle coordinates and the associated user's email field.

An authenticated non-owner user could send a request for a vehicle UUID that does not belong to their account and receive that vehicle's coordinates and account email field.

In a real application, this pattern could create privacy and safety risk because location data and account identifiers cross user boundaries. This sample rates severity as Medium because exploitation requires an authenticated user and knowledge of a valid vehicle identifier, while the exposed data is sensitive enough to require access control.

In CVSS terms the confidentiality impact is rated Low because each request exposes a single object's data rather than broad data access; the exposed fields are nonetheless sensitive (precise vehicle coordinates and an account email), which is why server-side access control still matters. The demonstrated exposure covers a single known vehicle object, not bulk disclosure, enumeration, or continuous tracking. The severity assumes a valid vehicle UUID is already known or obtainable; UUID discovery and enumeration were not tested in this review.

Severity factor	Assessment
Access required	Authenticated normal user
Attack complexity	Low after a valid vehicle UUID is known
User interaction	None
Confidentiality impact	Low (CVSS scope): one object's data per request, though the exposed coordinates and email are sensitive
Integrity impact	None observed
Availability impact	None observed

REPRODUCTION STEPS

1. Log in as the owner account and note the owner's own vehicle UUID, which is returned in the dashboard and the vehicle listing API response.
2. Send `GET /identity/api/v2/vehicle/{vehicleId}/location` with the owner's bearer token.
3. Confirm that the API returns HTTP `200` with `vehicleLocation`, `fullName`, and `email`.
4. Log in as a separate non-owner user account.
5. Repeat the same request using the non-owner user's bearer token and the same vehicle UUID.
6. Observe that the API again returns HTTP `200` with the vehicle location response instead of denying access.

EVIDENCE

Cleaned, redacted evidence was captured during testing and is available on request. The key request and response excerpts are reproduced below, and the sanitized application screenshots appear in the Methodology section above.

Evidence captured: `2026-06-08 13:34:37 KST`

Minimal redacted request shape:

```
curl -i \
 -H "Authorization: Bearer [REDACTED_TOKEN]" \
 "http://localhost:8888/identity/api/v2/vehicle/[REDACTED_UUID]/location"
```

The account boundary is summarized below; the raw redacted responses follow.

Request	Token subject	Requested vehicle owner	Expected	Actual
Control 1	None (no auth header)	n/a	`401`	`401`
Control 2	Owner	Owner (self)	`200`	`200`
Test 1	Non-owner A	Owner	`403` / `404`	`200`
Test 2	Non-owner B	Owner	`403` / `404`	`200`

Each of the three bearer tokens decodes to a distinct account subject (redacted): the owner, non-owner A, and non-owner B are three separate local accounts. Only the bearer token changed between Control 2 and Tests 1 and 2, while the requested vehicle UUID stayed constant. The identical `200` responses under different non-owner tokens are themselves the finding: ownership is never checked, so a non-owner token returns the owner's vehicle location and account email. Each `200` response returned `carId`, `vehicleLocation` (latitude/longitude), `fullName`, and `email`.

Key redacted evidence excerpt:

```
Control 1 - No Authorization Header
CRAPIResponse(message=Invalid Token, status=401)
HTTP_STATUS:401

Control 2 - Owner Account Token
{"carId":"[REDACTED_UUID]","vehicleLocation":{"id":
4,"latitude":"38.206348","longitude":"-84.270172"},"fullName":"Test","email":"[REDACTED_EMAI
L]"}
HTTP_STATUS:200

Test 1 - Non-owner Account A Token
{"carId":"[REDACTED_UUID]","vehicleLocation":{"id":
4,"latitude":"38.206348","longitude":"-84.270172"},"fullName":"Test","email":"[REDACTED_EMAI
L]"}
HTTP_STATUS:200

Test 2 - Non-owner Account B Token
{"carId":"[REDACTED_UUID]","vehicleLocation":{"id":
4,"latitude":"38.206348","longitude":"-84.270172"},"fullName":"Test","email":"[REDACTED_EMAI
L]"}
HTTP_STATUS:200
```

RECOMMENDATION

Enforce a server-side ownership check before returning vehicle location data. The handler should resolve the authenticated user's identity from the bearer token, look up the requested vehicle object, and return the location only if the vehicle belongs to that user or the caller has an explicit administrative permission.

The response should not include another user's email address unless the current workflow needs that field and the caller has authorization to receive it. For an authenticated caller without access to the requested vehicle, return a consistent denial response such as `403 Forbidden` or a non-enumerating `404 Not Found`.

RETEST GUIDANCE

After remediation, repeat the same request sequence:

Test case	Expected result
Owner token + owner vehicle UUID	HTTP `200`
Non-owner token + owner vehicle UUID	HTTP `403` or non-enumerating HTTP `404`
No authorization header + vehicle UUID	HTTP `401`
Denied response body	No vehicle owner, email, coordinates, or existence disclosure

Recommended regression coverage:

1. Add automated API tests for owner and non-owner access to `GET /identity/api/v2/vehicle/{vehicleId}/location`.
2. Add negative tests for random, deleted, and cross-account vehicle identifiers.
3. Verify that response serializers do not include account email fields unless the caller is authorized to receive them.

7. General Hardening Recommendations

These recommendations are general API hardening guidance for the local crAPI training scenario. They are not presented as confirmed issues unless supported by evidence from the local test instance.

Area	Recommendation
Object-level authorization	Enforce ownership checks server-side for every object read, update, and delete operation. Never rely on the client to hide object identifiers.
Function-level authorization	Map each privileged endpoint to an explicit role or permission check. Verify that non-privileged users receive consistent denial responses.
Object property exposure	Return only fields required by the client workflow. Use response DTOs or serializers that deny sensitive properties by default.
Authentication tokens	Keep tokens short-lived where practical, bind refresh-token rotation to server-side state, and avoid exposing tokens in logs, URLs, or browser storage where unnecessary.
Error handling	Return generic error messages to the user while logging diagnostic detail server-side. Avoid stack traces, internal hostnames, debug fields, and framework details in API responses.
Auditability	Log security-relevant access denials and ownership-check failures with safe identifiers that do not expose secrets or full personal data.
Regression testing	Add authorization regression tests for each confirmed BOLA, IDOR, or role-boundary issue after remediation.

8. Limitations

This report is based on a small, time-boxed review of a local training application. It does not represent a full penetration test, source code audit, infrastructure review, or production security assessment. Vulnerability discovery is not guaranteed in real client engagements. The value of a review includes the tested scope, methodology, findings if any, remediation guidance, and clearly documented limitations.

Additional limitations:

Limitation	Effect
Local training target only	Results do not describe a real client or production assessment
Small time-box	The sample includes one verified issue
No source code audit	Findings rely on black-box API behavior, not full implementation review
No destructive testing	The review excluded DoS, brute force, credential stuffing, and destructive workflows
Limited role coverage	The confirmed finding used normal authenticated user accounts; broader admin and mechanic role coverage was not completed
Limited endpoint coverage	This sample does not claim full coverage of crAPI's API surface
No identifier enumeration testing	Vehicle UUID discovery and enumeration were not assessed; the finding assumes a valid identifier is known or obtainable

9. Closing Notes

This review focused on verified depth over breadth: one confirmed finding, reproduced by hand, with concrete remediation and explicit limitations. AI-assisted candidate analysis was kept separate from manually verified findings, and no unverified candidate was presented as a confirmed issue.

A full engagement would extend the same workflow across additional issue classes, for example excessive data exposure or broken function-level authorization, with each issue held outside the confirmed findings until manual reproduction and cleaned evidence support it.

Appendix A — Severity Rating Scale

Severity reflects realistic impact and exploitation prerequisites for the observed behavior, aligned to CVSS v3.1 score bands. This sample contains one confirmed Medium-severity finding.

Severity	CVSS v3.1	Definition
Critical	9.0 – 10.0	Direct system compromise or full data exposure; exploitation typically trivial.
High	7.0 – 8.9	Significant impact on confidentiality, integrity, or availability; minimal prerequisites.
Medium	4.0 – 6.9	Limited impact or moderate prerequisites; may chain with other issues.
Low	0.1 – 3.9	Minimal direct risk; hardening recommendation.
Info	0.0	No direct security impact; best-practice or defense-in-depth note.

About the Researcher

Kyunghwan Byun

Web & API Security Researcher · YesWeHack Hunter

Focus areas: broken access control (OWASP API1 / API5), authentication and session flaws, IDOR / BOLA, and sensitive data exposure in web applications and REST APIs. My review workflow pairs AI-assisted endpoint analysis with manual reproduction and verification, and reports only issues confirmed by hand.

Methodology references: OWASP API Security Top 10 (2023), OWASP Web Security Testing Guide.

Available for small, permission-based Web / API security reviews — defined scope, written authorization, developer-friendly reporting.

EMAIL byunkh02@gmail.com GITHUB github.com/Koreahwan
YESWEHACK yeswehack.com/hunters/hwanwah LINKEDIN linkedin.com/in/kyunghwan-byun
UPWORK upwork.com/freelancers/~01dc11623939553de5